

CCT-1222 - Enforcement Hearings - API Solution - Analysis

Created by Sachin Dangui, last updated on 27/Apr/26 • 6 minute read

Resources

- Enforcement (Hearing Req) API Data Attributes v0.6 - Draft.xlsx
- api-cp-crime-caseingestion-prosecutor — CP-hosted OpenAPI spec for inbound enforcement endpoints
- cpp-context-staging-prosecutors — criminal staging-prosecutors context
- cpp-context-staging-prosecutors-civil — civil staging-prosecutors context

Decision log

- [@Jenny Taylor](#) confirmed that MCC is out of scope and enforcement cases will not be created using MCC.
- After having discussions with [@Julie Tyler](#), [@David Edwards](#) and [@Jenny Taylor](#), it was agreed that it makes sense to treat enforcement submissions as cases and not applications. 22 Apr 2026
- [@Debashis Sengupta](#) confirmed that there is no requirement to send out SPI OUTs to PNC for Enforcement cases.
- Authentication — shared-secret + IP whitelisting confirmed for both CCTs; OAuth 2 out of scope.** Decided 2026-03-09 (Richard Price Mathias email, "Re: RE: Authentication approach for Libra/CP and way forward for OPAL", to Kalyan Deevanapalli, Duncan Crawford, Steve Killelay, Andy Roberts; cc Dhevendran Natarajan). Both **CCT-1339** and **CCT-1222** will use shared secret + whitelisting — no OAuth 2 implementation within the scope of these CCTs. OAuth 2 work for Libra is deferred — candidate scope for **Project Opal**; to be added to the C&MI backlog for future discussion.

Under investigation

- Can we re-use one of the APIs of cpp-context-staging-prosecutors-civil to allow enforcement to create new cases of initiation type 0?
- Enforcement cases will always have civil offences (PSS) and not criminal offences (PNLD). At the moment cpp-context-prosecution-casefile has some logic to look up civil offences based on the CIVIL channel.

CP ↔ Libra API interactions (CCT-1222)

Sourced from the CCT-1222 sequence diagram on the [E2E Process page \(1864147959\)](#). Internal CP calls (S&L, Fix Case UX, Hearing, Third Party fan-out) are excluded — only CP↔Libra boundary calls are listed.

#	Direction	Method & Path	Operation	Hosted by	Sync / Async	Authentication	FR / CIMD	Notes
1	Libra → CP	POST /stagingprosecutors-civil/prosecutions/	Submit enforcement case	CP	Async — 202 Accepted with submissionId	APIM shared-secret 0cp-Apim-Subscription-Key + IP whitelisting (product <i>Libra</i> Access, sub <i>Libra User</i>)	FR01, FR03, FR04, FR05 · CIMD-3531	Interim: reuses existing 22). Initiation code 0 others-prosecut submission.v1+j /enforcementpro diagram and POST / Status table.
2	Libra → CP	GET /submissions/{submissionId}	Poll for submission status / case-created	CP	Sync	APIM shared-secret + IP whitelisting	FR09 · CIMD-3540	Returns submission : this move to GET /s civil/prosecuti for consistency with
3	CP → Libra	POST /cases	Confirm case scheduled (hearing successfully allocated)	Libra	Sync — ack	Libra-controlled — TBC (routed via cpp-gob-gateway-service)	FR09 · CIMD-2862	Replaces earlier Libr HearingConfirmed Al
4	Libra → CP	PUT /enforcementprosecutions	Update existing case hearing (from S&L UX update)	CP	Sync — ack	APIM shared-secret + IP whitelisting	FR11 · CIMD-2934	Diagram draws this e Status treats as Libr /prosecutions2. endpoint, should the
5	CP → Libra	PUT /cases	Send case / hearing update to GoB	Libra	Sync — ack	Libra-controlled — TBC	FR11 · CIMD-2934	Aligns to Libra Heari
6	CP → Libra	POST /cases/results	Send hearing result to GoB and request NOWS data in same call	Libra	Synchronous — 200 body = HearingResultedResponse / NOWSDataItems	Libra-controlled — TBC	FR14 (+ FR13) · CIMD-2864 (+ CIMD-2970)	Request body = Hea response body = Hea produces NOWs inlr separate callback en
7	CP → Libra	GET (path TBD)	Generate / retrieve outstanding fines report	Libra	Sync	Libra-controlled — TBC. Libra-side already exists	FR17 · CIMD-2940	Libra returns fine infc

- C&MI → Fix Case UX — business-verification failure queue
- C&MI → Third Party — case resulted fan-out

Related — CCT-1339 interactions

#	Direction	Method & Path	Operation	Hosted by	Sync / Async	Authentication	FR / CIMD	Notes
8	CP → Libra	POST /fine-account	Create fine account	Libra	Sync — ack	Libra-controlled — TBC	CIMD-2842 (+ FR15 CIMD-2938)	Triggers Libra-side account creation.
9	Libra → CP	POST /enforcementprosecutions/fineaccount	Receive fine-account information into CP	CP	Sync — ack	APIM shared-secret + IP whitelisting (product <i>Libra Access</i> , sub <i>Libra User</i>)	CIMD-2842	End-state auth for CCT-1339.

Authentication summary

- **Libra** → **CP** (inbound via CP's APIM): APIM shared-secret header `0cp-Apim-Subscription-Key` + IP whitelisting. APIM product *Libra Access*, subscription *Libra User*. End-state for both CCT-1339 and CCT-1222 (see Decision log entry 4).
- **CP** → **Libra** (outbound via `cpp-gob-gateway-service`): auth defined by Libra, not documented on the E2E page. Marked *TBC* on outbound rows until Libra confirms.

Case-creation APIs — side-by-side

CPPI and Civils case creation interfaceCPPI and Civils case creation interface2

Endpoints

Aspect	cpp-context-staging-prosecutors (criminal) [CPPI]	cpp-context-staging-prosecutors-civil [CPCI] → cpp-context-staging-civil-cases
Base path	POST /command/api/rest/stagingprosecutors/v2/prosecutions/{oucode}	POST /command/api/rest/stagingprosecutors-civil/prosecutions/ --> /command/api/rest/stagingprosecutors-civil/cases/
Content types	sjp-prosecution.v2+json, summons-prosecution.v2+json, charge-prosecution.v2+json, requisition-prosecution.v2+json	stagingprosecutorscivil.charge-prosecution+json -->. Should be renamed to stagingprosecutorscivil.others-prosecution+json, stagingprosecutorscivil.summons-prosecution+json
Extra endpoint	POST /SubmitApplication (stagingprosecutors.submit-application+json)	—
Response	202 Accepted with UrlResponse JSON	202 Accepted with UrlResponse JSON
RAML location	stagingprosecutors-command-api/src/raml/staging-prosecutors-command-api.raml	stagingprosecutorscivil-command-api/src/raml/staging_prosecutors_civil_command_api.raml

Commands and handlers

Media type	Command type	Initiation code
sjp-prosecution.v2+json	stagingprosecutors.command.sjp-prosecution	J
summons-prosecution.v2+json	stagingprosecutors.command.summons-prosecution	S
charge-prosecution.v2+json	stagingprosecutors.command.charge-prosecution	C
requisition-prosecution.v2+json	stagingprosecutors.command.requisition-prosecution	Q
stagingprosecutorscivil.charge-prosecution+json	stagingprosecutorscivil.command.charge-prosecution	0 (hardcoded in ProsecutionCaseToGroupProsecutionConverterForCharge)
stagingprosecutorscivil.summons-prosecution+json	stagingprosecutorscivil.command.summons-prosecution	S (hardcoded in ProsecutionCaseToGroupProsecutionConverterForSummons)

Downstream: the civil event processor (`ProsecutionChargedEventProcessor`) translates the received event into `InitiateProsecution` / `InitiateGroupProsecution` commands sent to the prosecution case-file context, with `withIsCivil(true)` set on the group prosecution.

Submission cardinality

- **CPPI (criminal)**: 1 submission = 1 case (with 1..N defendants). No wrapper to submit multiple cases in one request.
- **Civil**: 1 submission = 1..N cases, all sharing one prosecutingAuthority and one hearingDetails at the top level. Supports both:
- **Individual / single civil case** — `prosecutionCases` with a single element, which may contain multiple defendants.
- **Bulk civil cases** — `prosecutionCases` with many elements, each containing exactly one defendant.
- `caseMarker` has the same per-case semantics in both — CPPI places it on the submission-details object (which is 1:1 with the case), civil places it inside each `prosecutionCases[]` element.

2. **Initiation code.** Criminal supports J / S / C / Q; civil supports 0 (charge) and S (summons). Initiation type is determined by the endpoint/content-type chosen, not a data field — the client picks the path, and the converter hardcodes the code.
3. **Channel.** Civil forces Channel.CIVIL and isCivil = true in the converters; criminal does not.
4. **Offence-type distinction.** Neither repo carries a PSS vs PNLD selector on the request. Both only carry cjs0ffenceCode. The civil-channel / PSS lookup happens downstream in cpp-context-prosecution-casefile.
5. **Endpoint surface area.** Criminal has four prosecution variants plus /SubmitApplication; civil has only charge + summons.

Implications for CCT-1222

- The civil **charge** endpoint is the closest existing fit: it already emits initiation code 0 and sets isCivil = true, matching the enforcement case profile.
- PSS/PNLD is not differentiated at the intake layer in either repo; that distinction is resolved further down by cpp-context-prosecution-casefile using Channel.CIVIL.

Payload differences across case types

The six existing intake payloads differ in which blocks and fields are present. Highlights only — field-level detail is in the source RAML / JSON schemas.

Payload feature	SJP (J)	Summons (S)	Charge (C)	Requisition (Q)	Civil Charge
Hearing block in intake	✗	✓	✓	✓	✓ hearing (court + date)
Multiple defendants per case	✗ single defendant	✓ defendants []	✓ defendants []	✓ defendants []	✓ defend
Multiple cases per submission	✗	✗	✗	✗	✓ prosecuti
Root shape	{prosecutionSubmissionDetails, defendant}	{prosecutionSubmissionDetails, defendants []}	same as S	same as S	{prosecution prosecutingA hearingDe
writtenChargePostingDate	✓ required	✗	✗	✗	✗
Offence: laidDate required	✗	✗	✗	✗	✓
paymentReference on case	✗	✗	✗	✗	✓
Offence requires offenceCommittedDate + offenceDateCod	✗	✓	✓	✓	✗ (optional)

Headline differences:

- Only SJP has no hearing block at intake; SJP is also the only single-defendant case type and the only one that requires writtenChargePostingDate.
- Only civil supports bulk (multiple cases per submission via prosecutionCases[], all sharing one root-level hearingDetails).
- Only civil requires laidDate on each offence; only criminal S/C/Q require offenceCommittedDate + offenceDateCode.
- paymentReference is civil-only. alcoholRelatedOffence and offence-level chargeDate are criminal-only.

Query log

Questions raised with BA / Product Owner during CCT-1222 analysis.

#	Raised	Question	Raised with	Status	Answer
1	2026-04-15	In cpp-context-prosecution-casefile, offence-code enrichment (OffenceDataRefDataEnricher) drives the offence reference-data lookup off the case's isCivil flag: civil cases search both PSS (civil offences) and PNLD (criminal), non-civil cases search PNLD only. Is the civil-vs-non-civil split a deliberate business rule or is it historical? If it isn't a hard rule, dropping the branch so every case searches both catalogues would remove a civil-vs-criminal routing concern.	BA / PO	Open	—

No labels

1 Comment



Debashis Sengupta
22/Apr/26

ta confirmed that there is no requirement to send out SPI OUTs to PNC for Enforcement cases.

I have queried about the SPI OUT requirement for the Enforcement case with Jenny and Julie multiple times. There is no such requirement. Also the initial Enforcement Prosecution reference data setup, Jenny as set the SPI OUT flag as false and the same setup has been reviewed by Enforcement team. Further more Julie explained that PNC team only requires SPI OUT for the recordable offence and current Enfocement offence list does not have any offences in referencedata_moj_offences where this flag is true (or "Y"). Hence the final conclusion as of today is there is no requirement for SPI OUT for the Enforcement case hearing results.